Assignment 2: Tracing Lab

CYBERSEC590 report. Due Wed Sep 30, 2026, 11:59 pm

Field	Entry
Name	Dan Hendricks, Sera Tan, Justin Abraham, Shweta Mittal
Provider	Ollama
Exact model ID	gemma4:31b
Langfuse region / base URL	"https://us.cloud.langfuse.com"
Date of runs	09-23-2026

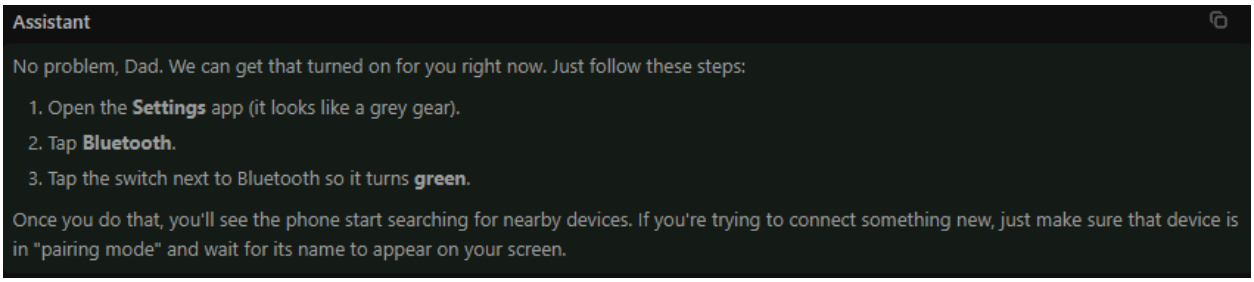
Question 1 (Dan)

Setup and baseline: Which provider and exact model ID did you use? Did setup, the provider probe, and the initial app question succeed? Why is no Langfuse trace expected before you implement the lab, even if the project keys are populated?

Checkpoint	Result (succeeded / failed, with detail)
Setup (uv run lab.py setup)	success
Provider probe (uv run lab.py probe)	success
Tool calling in probe	success
Initial app question	success

Bluetooth baseline question: "How do I turn Bluetooth on my iPhone?"

App's actual answer:



Why no trace is expected before implementation: **No tracing is expected during this step because although we have valid Langfuse keys, we have not configured the exporter to send any data to the API.**

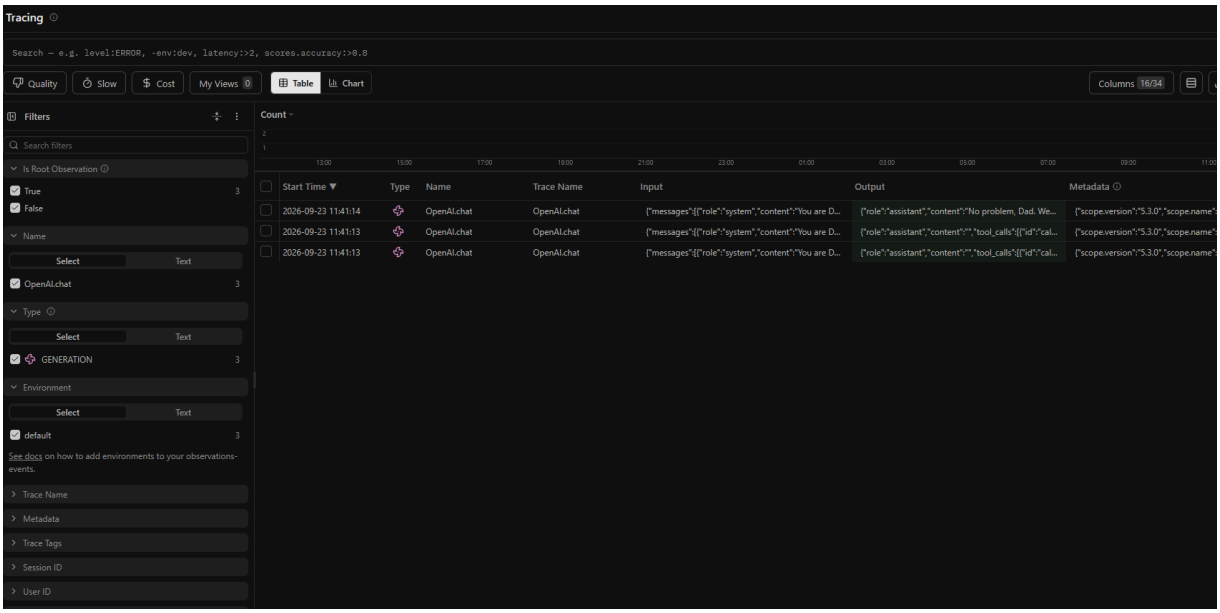
Question 2 (Shweta)

Implementation: What changes did you make in index.ts, support-agent.ts, and tools.ts? Explain how these changes enable model generations, the agent parent, and actual tool observations. What does the exporter's flush/shutdown logic do?

File	What I changed	How it enables tracing
index.ts	Initialized the Langfuse exporter and registered it with the OpenTelemetry NodeSDK (via LangfuseSpanProcessor), gated behind isLangfuseConfigured() so it only starts when both LANGFUSE_PUBLIC_KEY and LANGFUSE_SECRET_KEY are present.	This is the pipeline that actually ships spans to Langfuse. Without this step, every observe()-wrapped call in the other two files still runs, but the spans have nowhere to go and are silently dropped, which is exactly the failure mode documented in Problem 3.
support-agent.ts (Step 1 and Step 2)	Step 1: wrapped the model client call so every LLM invocation is captured as its own span with input/output, model name, and provider metadata. Step 2: wrapped the whole per-turn handler in an observe() call named dad-it-support-chat-turn, so it becomes the parent span that all generations and tool calls nest under.	Step 1 produces the GENERATION observations (the OpenAI.chat entries). Step 2 produces the single AGENT observation per turn, giving the trace its tree structure instead of a flat list of unrelated spans.
tools.ts (Step 3)	Wrapped get_support_context and search_help_library themselves (not just the dispatcher) in observe(), so calling the real function body, with its actual arguments and actual return value, it gets logged.	This is what turns a tool *request* in the model's output into a real, observed TOOL span. Because the observed wrapper sits on the function that actually runs, the trace reflects what really executed, not just what the model asked for.

Flush / shutdown logic:

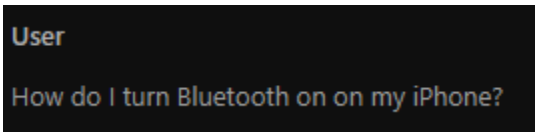
Spans are exported to Langfuse in batches, asynchronously, so they can sit in memory briefly before being sent. The flush/shutdown logic forces any buffered spans to be sent immediately when the process is stopped or restarted (uv run lab.py restart), so a shutdown doesn't silently drop the last batch of trace data before it reaches the Langfuse API



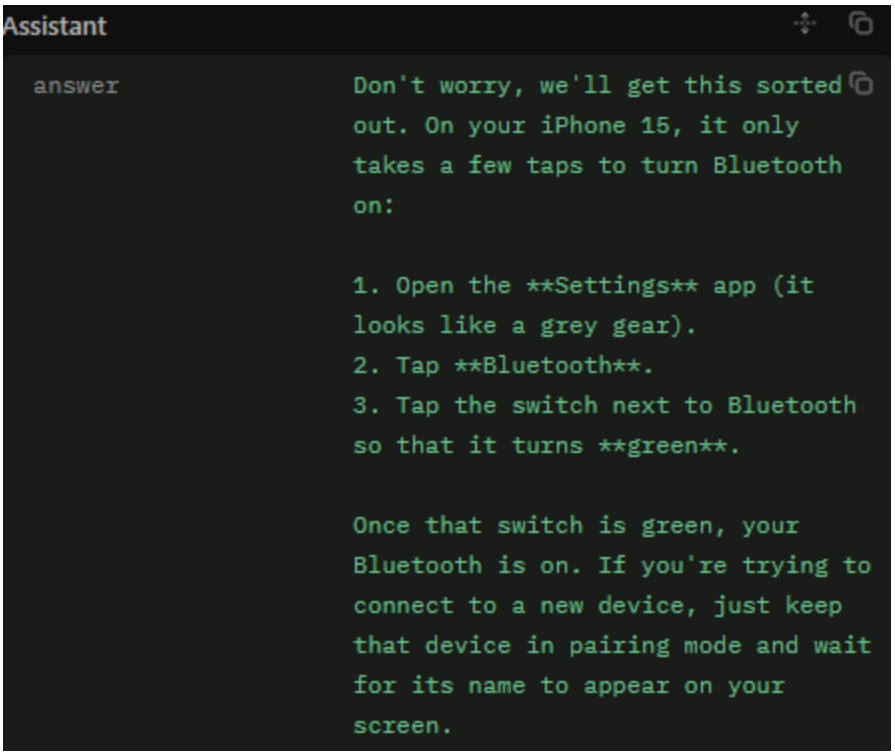
Question 3 (Shweta)

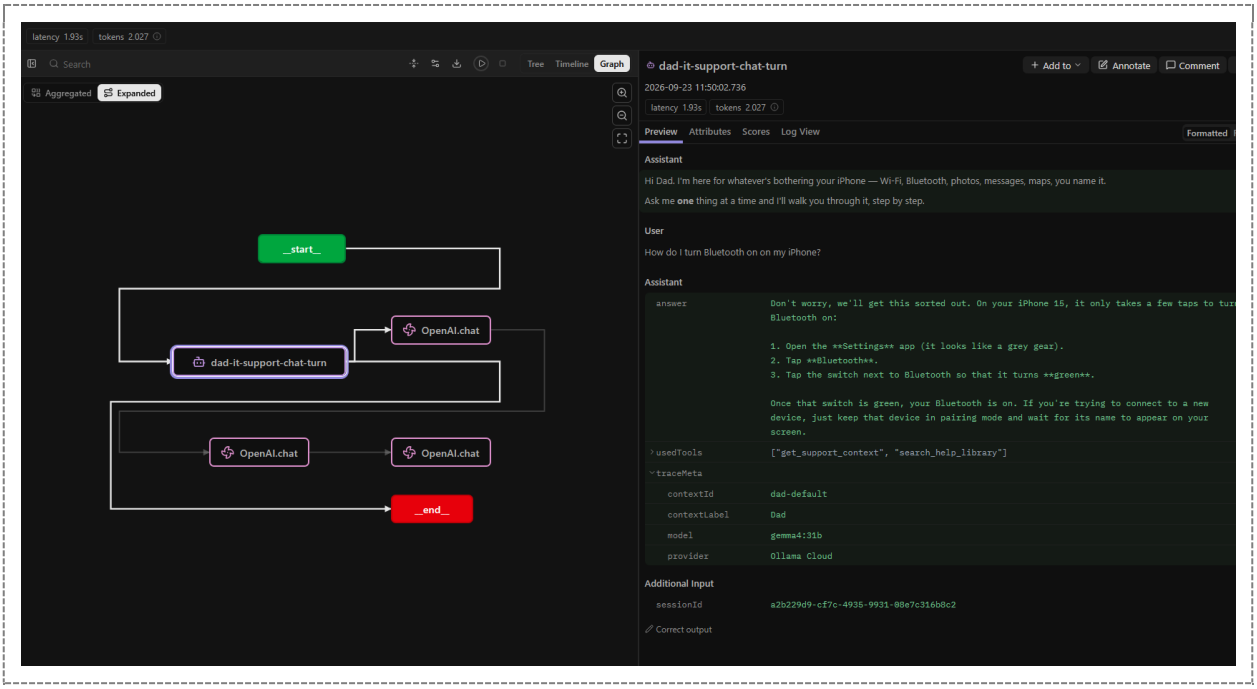
Trace evidence: Provide the Bluetooth question and the app's actual answer. Include a credential-free screenshot of the expanded matching trace. Identify the AGENT parent, GENERATION observations, and any TOOL observations. How is a tool request in a model response different from an observed tool execution?

Question asked:



App's actual answer:





Observation type	Name(s) seen	Notes (order, count)
AGENT parent	dad-it-support-chat-turn	1 per turn, the root span everything else nests under

GENERATION	OpenAI.chat	3 total, first generation decides to call the two tools, a second/third generation processes each tool's result before producing the final answer text
TOOL	get_support_context, search_help_library	2 total, one each, both nested directly under the AGENT parent, run after the first generation and before the final generation

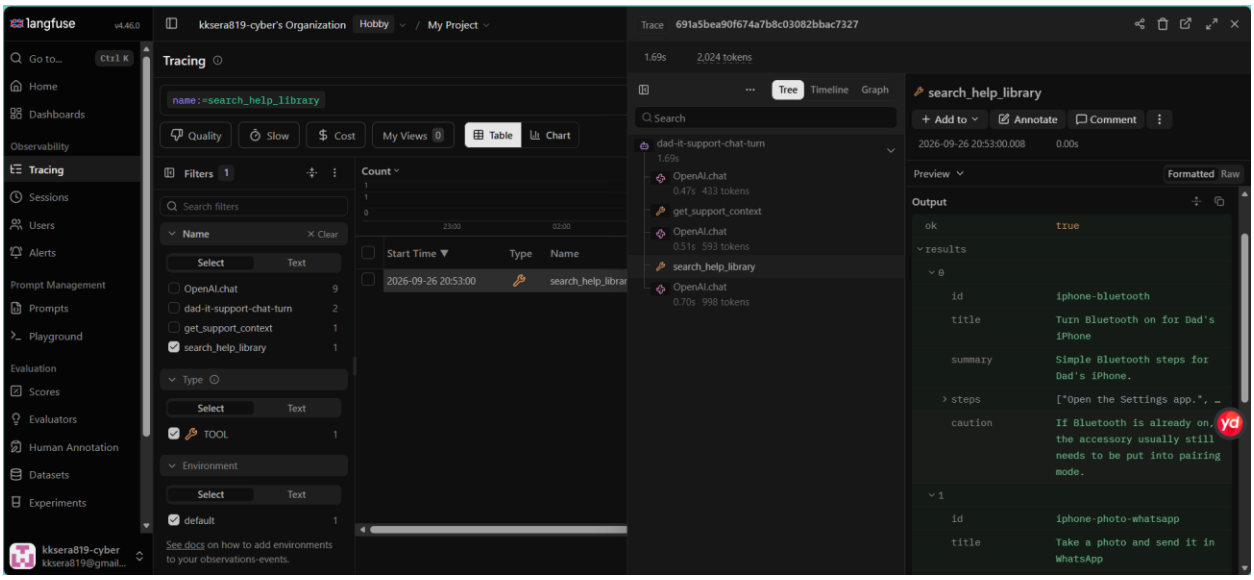
Tool request vs. observed tool execution: In the first GENERATION's output, the model's response includes a tool_calls field, this is just the model *declaring intent* to call a tool with certain arguments. it hasn't actually run yet. The TOOL observation is a separate span created only when the dispatch function actually invokes the real get_support_context/search_help_library code and logs its real return value. This distinction matters because a model can request a tool call that never executes, only the presence of an actual TOOL span in the trace proves the tool ran and produced real data.

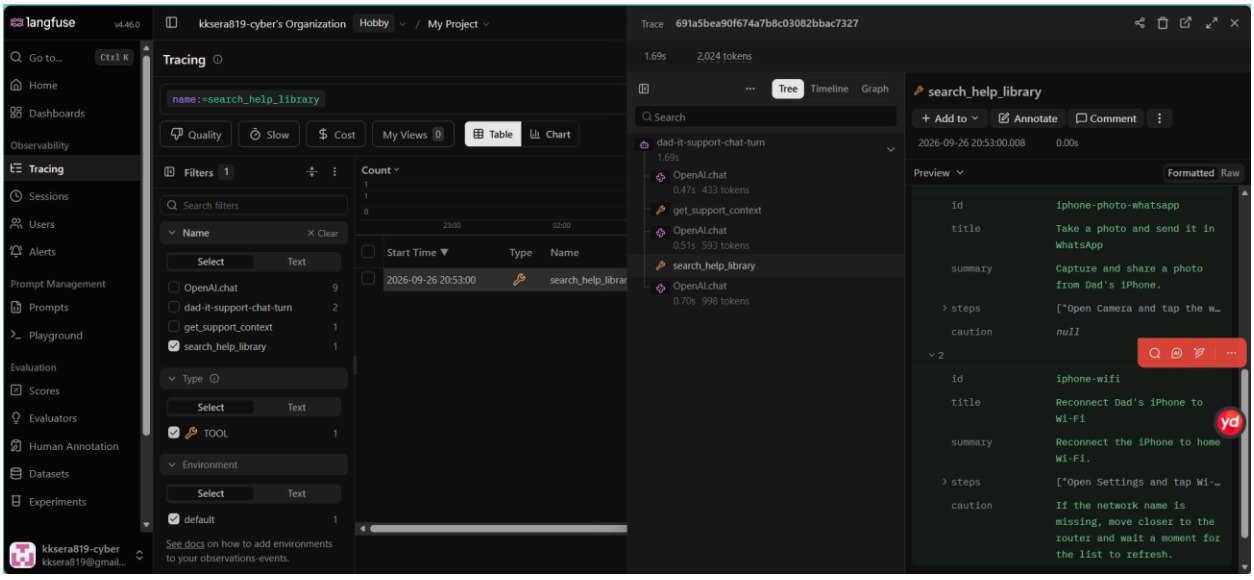
Question 4 (Sera)

Evidence in the answer: Identify a specific device-context detail and a specific help-guide detail in the tool results. Explain whether and how the final answer uses each detail. If the model ignored evidence or skipped a tool, document that instead of assuming it used the evidence.

Evidence	Exact detail from tool result	Used in final answer? How?
Device context (get_support_context)	iPhone 15	Yes. The final answer explicitly says “on your iPhone 15,” showing that the model used the device context.
Help guide (search_help_library)	titled “Turn Bluetooth on for Dad’s iPhone.” Its steps include opening the Settings app, and its caution says that an accessory may still need to be put into pairing mode.	Yes. The final answer tells Dad to open Settings, tap Bluetooth, turn the switch green, and put a new accessory into “pairing mode.”

Ignored evidence or skipped tools: None observed. Both get_*support_context and search_help_*library were executed, and the final answer used details from both tool results.





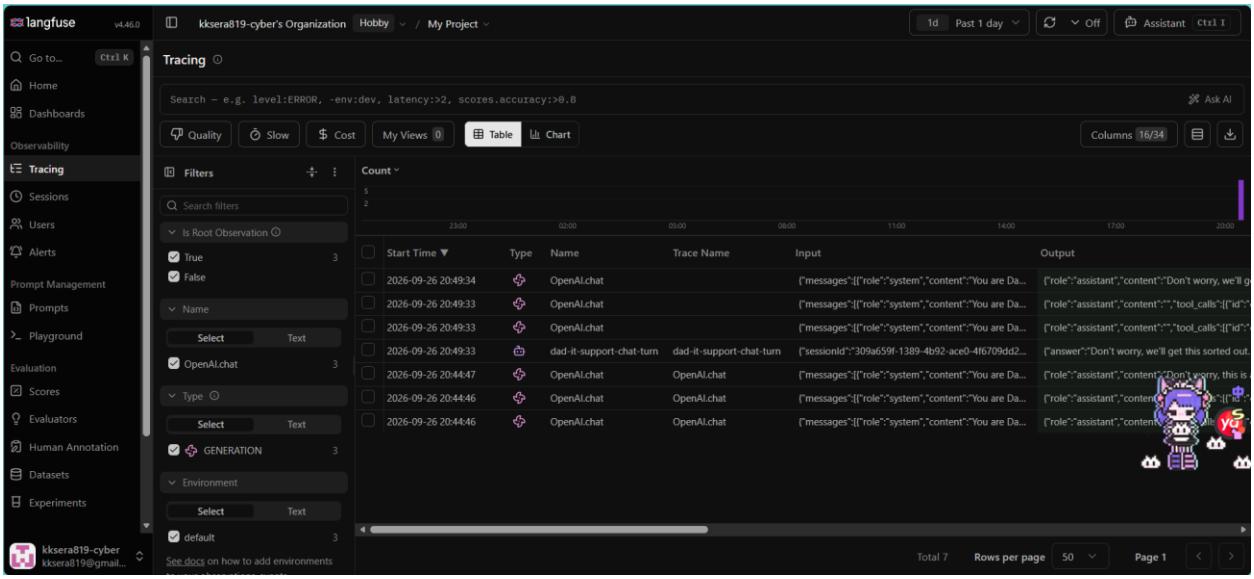
Question 5 (Sera)

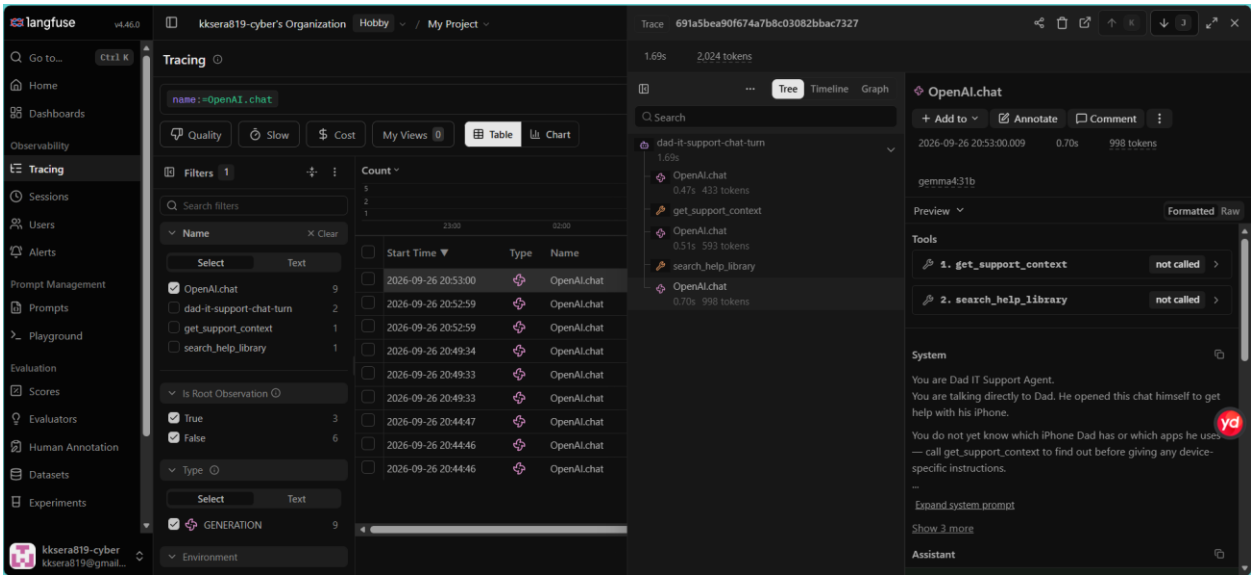
Timing and usage: Which observed step took the longest, and what was its duration? Report token usage where available and identify whether it is for a generation or the whole turn. Is cost shown? Explain why missing cost data cannot be treated as zero cost.

Item	Bluetooth trace	Notes / scope
Longest step (name)	OpenAI.chat	Third GENERATION
Longest step duration	0.70s	Generation-level duration
Total turn duration	1.69s	Whole dad-it-support-chat-turn trace

Input tokens	Not shown	not visible in the observed Langfuse view
Output tokens	Not shown	not visible in the observed Langfuse view
Total tokens	2024 tokens	Whole trace total
Cost shown?	Not shown	No cost value was displayed

Why missing cost is not zero cost: Because it only means that Langfuse did not have pricing or cost metadata available for this model/provider. The model request may still have incurred usage or provider charges.



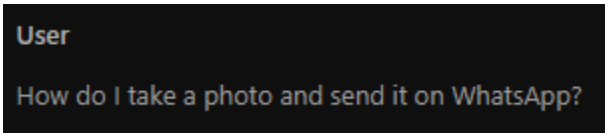


Question 6 (Dan)

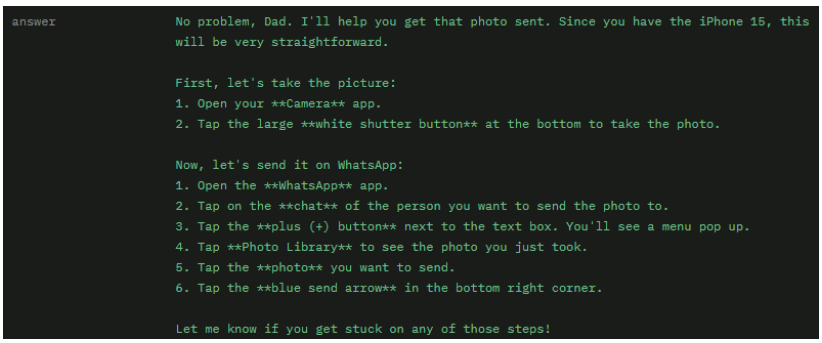
Second question: Provide your WhatsApp or Wi-Fi question and the app's actual answer. What changed in the help-library results compared with Bluetooth? Did the tool calls or generation count change? Support your comparison with the two traces.

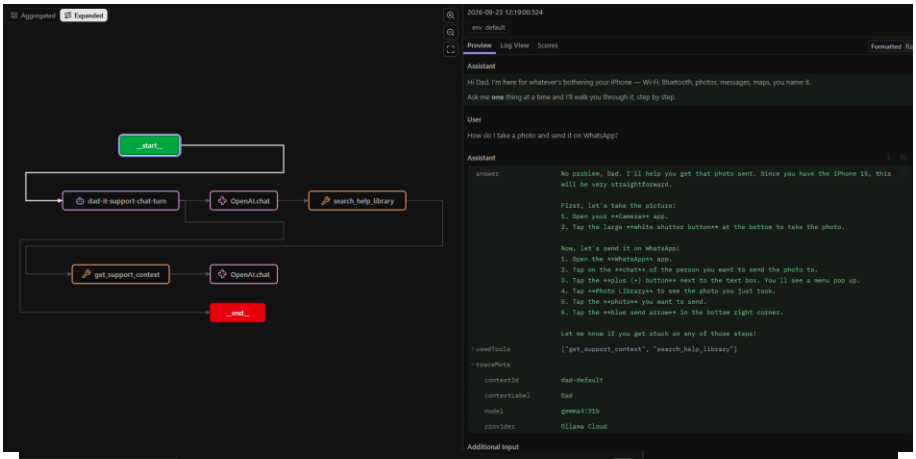
The number of tool and generation calls remained the same for the 2 calls; the only differences I saw were in the search libraries that were used and the final token usage.

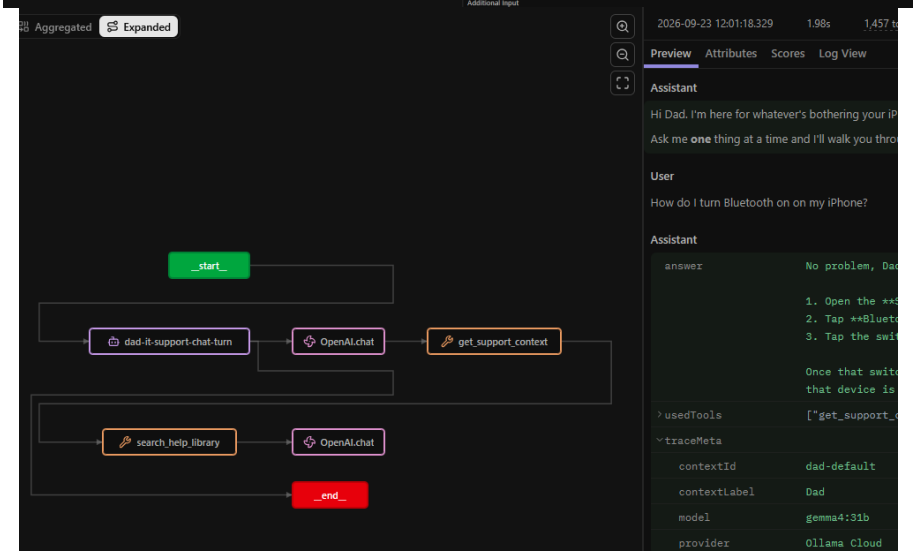
Second question asked:



App's actual answer:







Comparison	Bluetooth trace	Second trace
Help-library result title(s)	iphone-bluetooth "iphone-photo-whatsapp" "iphone-wifi"	iphone-photo-whatsapp iphone-maps-bus-stop iphone-photo-preview
Tool calls (names, count)	2	2
Generation count	2	2
Longest step / duration	OpenAI.chat / 1.17s	OpenAI.chat / 1.26s
Token usage	1457 total	1537 total

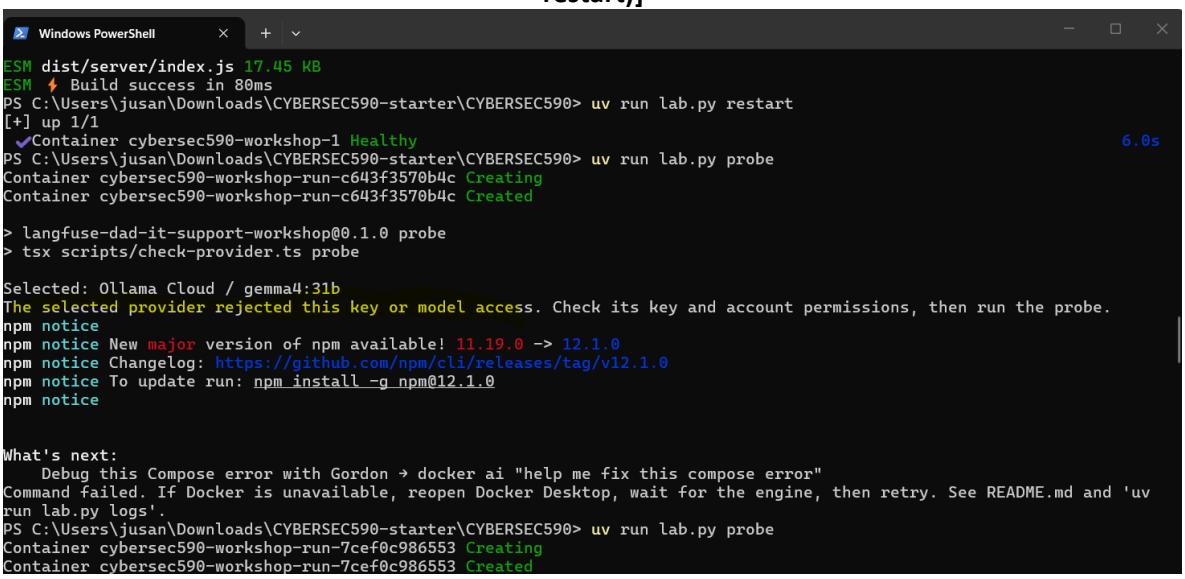
Question 7 (Justin)

Problems and diagnosis: What setup, trace-delivery, or model-behavior problems did you encounter? Explain the evidence, what you checked or changed, and the outcome. If you encountered none, explain what you would check if the app answered successfully, but no trace appeared after implementation.

Problem	Evidence (error, output)	What I checked or changed	Outcome
1. The app could not call the model: provider API key missing (setup)	Asking "How do I turn Bluetooth on on my iPhone?" in the app at http://127.0.0.1:3333 returned: "Set OLLAMA_API_KEY in workshop/.env and restart the app." No answer was produced. See Screenshot A.	MODEL_PROVIDER=ollama, so the app only reads OLLAMA_API_KEY. The Duke key field is ignored. I created a key at ollama.com/settings/keys and pasted it into OLLAMA_API_KEY in workshop/.env with no quotes or spaces. I confirmed the file is named .env, not .env.txt. I ran uv run lab.py probe then uv run lab.py restart, and reloaded the app.	The Bluetooth question returned an answer and the header changed to "last request succeeded." This became my Question 1 baseline. As expected, Langfuse had no trace before implementation.
2. Langfuse credential hygiene and a malformed base URL (trace delivery)	Langfuse's in-app assistant reported: "Your project hasn't received any traces in the last day or so." It flagged that the key pair had been pasted into a chat (exposed) and that LANGFUSE_BASE_URL had a leading space. See Screenshot B.	Deleted the exposed key pair under Settings → API Keys and created a new pair. Edited workshop/.env so the line reads exactly LANGFUSE_BASE_URL=https://us.cloud.langfuse.com (no leading space, plain URL, US region matching my project).	The exposed keys can no longer be used, and the base URL matches my project's region. Lesson: keys go only in the server-side .env file, never in chats, screenshots or the report.
3. The app answered, but no trace appeared after implementat	After adding the tracing code, the Bluetooth question succeeded in the app, but Tracing → Traces in	Checked in order: the tracing code was in all three files and uv run lab.py check passed; the correct project and time range were selected in Langfuse; then workshop/.env. LANGFUSE_PUBLIC_KEY and LANGFUSE_SECRET_KEY were not filled in. In index.ts the exporter is created only	A dad-it-support-chat-turn trace appeared, with the AGENT parent, GENERATION children and the

ion (trace delivery)	Langfuse stayed empty after refreshing. See Screenshot C (before).	when isLangfuseConfigured() is true, so without both keys no LangfuseSpanProcessor or NodeSDK started and every span was dropped. I added the new project keys, ran uv run lab.py restart and asked a new question.	get_support_context and search_help_library TOOL observations. See Screenshot C (after). A successful chat does not prove trace delivery; only the matching trace in Langfuse does.

Screenshot A — **[SCREENSHOT: the app showing "Set OLLAMA_API_KEY in workshop/.env and restart the app." (to recreate: blank OLLAMA_API_KEY, restart, ask, then restore the key and restart)]**



Screenshot B — **[SCREENSHOT: Langfuse's assistant message "Your project hasn't received any traces…", with any key text cropped out]**

```
 langfuse-dad-it-support-workshop@0.1.0 build:client
 vite build

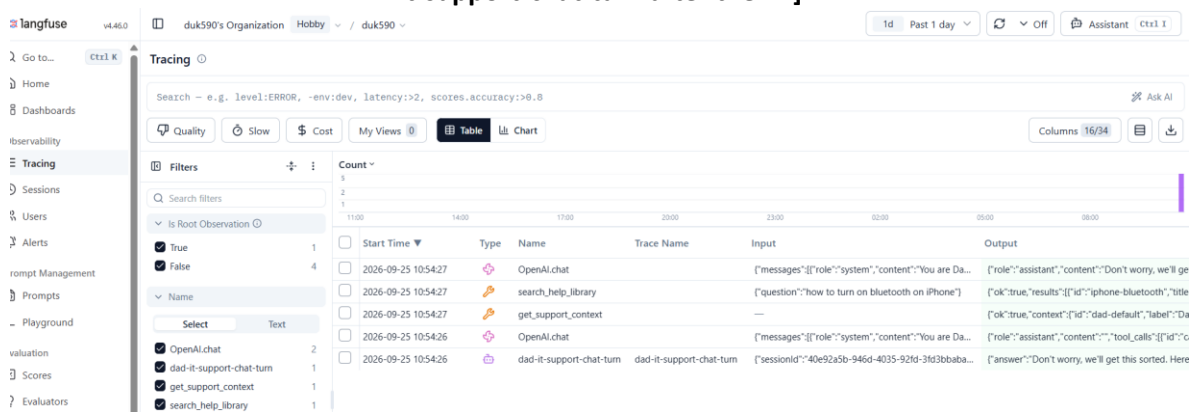
ite v8.0.12 building client environment for production...
 266 modules transformed.
omputing gzip size...
ist/index.html                   0.49 kB │ gzip:   0.31 kB
ist/assets/index-Dt8XSAij.css   20.73 kB │ gzip:   4.75 kB
ist/assets/index-BXTC7RbC.js   354.51 kB │ gzip: 108.77 kB

 built in 372ms

 langfuse-dad-it-support-workshop@0.1.0 build:server
 tsup src/server/index.ts --format esm --out-dir dist/server --clean

LI Building entry: src/server/index.ts
LI Using tsconfig: tsconfig.json
LI tsup v8.5.1
LI Target: es2022
LI Cleaning output folder
SM Build start
SM dist/server/index.js 17.45 KB
SM ⚡ Build success in 88ms
S C:\Users\jusan\Downloads\CYBERSEC590-starter\CYBERSEC590> uv run lab.py restart
+] restart 0/1
- Container cybersec590-workshop-1 Restarting
+] up 1/1
✔Container cybersec590-workshop-1 Healthy
S C:\Users\jusan\Downloads\CYBERSEC590-starter\CYBERSEC590>
```

Screenshot C — **[SCREENSHOT: Tracing → Traces empty before the fix, then the list showing dad-it-support-chat-turn after the fix]**



If no problems: what I would check if the app answered but no trace appeared: Not applicable, because I hit the problems above. The checklist I used, in order: (1) LANGFUSE_PUBLIC_KEY and LANGFUSE_SECRET_KEY both filled in, from the same project, and not the rotated pair; (2) LANGFUSE_BASE_URL is a plain URL with no spaces or Markdown, matching the project region (US vs EU); (3) uv run lab.py restart after every .env change, then a new question in the app (the probe never creates traces); (4) the tracing code present in index.ts, support-agent.ts and tools.ts, and uv run lab.py check passing; (5) uv run lab.py logs for export errors (401 = wrong keys, connection/DNS errors = wrong URL or no network); (6) in Langfuse, the correct project, a wide enough time range, and a refresh a few seconds after the answer, because spans are batched.